Lightworks: architecture and folder layout

A FreeRTOS beginner's guide to this repo

11 November 2026

The big picture

Lightworks is firmware for a NUCLEO-H743ZI2 board (STM32H743, a Cortex-M7) that drives a strip of 144 WS2815 addressable LEDs. Right now it shows a dim green on the whole strip, 30 times a second, and blinks the yellow LED (LD2) once a second so you can see it's alive.

The whole design rests on one idea: code that knows about LEDs is kept apart from code that knows about this chip. That way the project can move to another microcontroller later by adding a new port folder, without touching the LED logic.

So the folders are layers, and each one only uses the layers below it:

```
apps/nucleo_h743zi2      what this product does: tasks, priorities, colours
      |         \
      v          v
core/include/    ports/stm32h743
lightworks       everything chip-specific: boot, clocks, memory map, SPI+DMA driver
(pure LED          |
 logic, no         v
 hardware)       third_party
                 FreeRTOS kernel, ARM CMSIS and ST register headers
```

The port also uses core (the driver encodes core pixels), and the app uses FreeRTOS directly to create its tasks.

The folders

core/include/lightworks/

The product logic, with no hardware in it.

- `rgb.hpp` defines a pixel (red, green, blue bytes).
- `span.hpp` is a pointer-plus-length view of an array, a stand-in for C++20's `std::span` since the project is C++17. It doesn't own memory.
- `frame_buffer.hpp` holds two frames. The effect draws into the back frame while the driver sends the front one, and `publish()` swaps them.
- `ws2815.hpp` turns pixels into the strip's bit pattern (green, red, blue order, most significant bit first) and checks the datasheet timings at compile time. If the SPI clock is changed to one that can't produce valid WS2815 pulses, the build fails.
- `control.hpp` is an empty placeholder where serial, Wi-Fi, sensors or OTA updates will plug in later.

Because none of this touches registers, `tests/` can compile and test the encoder on a laptop with the normal desktop compiler.

ports/stm32h743/

Everything that only makes sense on this chip.

- `startup/startup_stm32h743xx.s` is the assembly that runs at reset, plus the vector table: the list of interrupt handler addresses.
- `linker/stm32h743zi.ld` is the memory map. It tells the linker where code and each kind of data lives.

- `src/clock.cpp` raises the clocks from the 64 MHz reset clock to a 240 MHz CPU.
- `src/runtime.cpp` supplies `memcpy`, `memset` and friends, and the loop that runs global constructors, because the firmware links no C library.
- `src/ws2815_spi_dma.cpp` is the actual LED driver (SPI1 plus DMA, see below).
- `src/status_led.cpp` drives LD2 (heartbeat) and LD3 (fault).
- `src/freertos_hooks.cpp` gives FreeRTOS memory for its idle task and handles stack overflow by lighting the red LED.
- `include/board.h` holds the pins, clock numbers and pixel count. `include/FreeRTOSConfig.h` configures the kernel.

apps/nucleo_h743zi2/

The thin top layer. `main.cpp` sets up the status LED and calls `app::run()`. `app.cpp` creates the FreeRTOS tasks and starts the scheduler.

third_party/

Code that wasn't written for this project: the FreeRTOS kernel (a git submodule), ST's STM32H7 device headers (a submodule) and ARM's CMSIS Core headers (copied in). The device and CMSIS headers give every register a name, such as `SPI1->CR1`.

Build files

`cmake/arm-none-eabi.cmake` tells CMake to use the ARM cross-compiler instead of the desktop gcc. The top `CMakeLists.txt` lists every source file that goes into `firmware.elf`, and turns the ELF into `firmware.bin` for flashing. `tests/CMakeLists.txt` is a separate desktop-only project.

What FreeRTOS adds

Without an operating system, firmware is one big loop doing everything in turn. FreeRTOS lets you write several independent loops, called tasks, and switches between them. The rule is simple: the highest-priority task that is ready always runs. A task that's waiting (on a delay or a semaphore) uses no CPU at all.

Lightworks has three tasks:

1. **status** (priority 1) toggles the yellow LED every 500 ms. It's a heartbeat.
2. **effect** (priority 2) wakes every 33 ms and fills the back frame with a colour. Once the strip isn't busy it swaps the frames and "gives" the `frame_ready` semaphore, which signals that a new frame is ready.
3. **output** (priority 3) sleeps on that semaphore. The moment it's given, this task runs straight away because it has the highest priority. It encodes the frame and starts a DMA transfer, so the hardware shifts the bits out over SPI while the CPU is free. Then it waits on a second semaphore that the SPI interrupt (`SPI1_IRQHandler`) gives once the last bit has left the pin.

This is the pattern you'll see in most RTOS firmware: an interrupt does the bare minimum and wakes a task, and the task does the real work.

If a transfer times out or a task overruns its stack, the red LED (LD3) comes on.

Two choices worth knowing about:

- **Static allocation only.** `FreeRTOSConfig.h` switches off dynamic allocation, so every task's stack and control block is a plain global array, created with `xTaskCreateStatic`. Memory use is fixed when you build, and it can't run out at runtime.

- **The CM4F port on an M7.** The build compiles FreeRTOS's generic `ARM_CM4F` port even though the chip is a Cortex-M7. That's normal: the two cores look the same to FreeRTOS. The SVC, PendSV and SysTick entries in the vector table point at that port's handlers, which is how FreeRTOS switches tasks.

How the LED data gets out

The WS2815 needs precisely timed pulses. Instead of toggling a pin from software, the driver uses SPI1 at 3 MHz, where eight SPI bits make one LED bit: `0x80` is a 0 and `0xC0` is a 1. After the last pixel it sends enough zero bytes to hold the line low for more than 280 µs, which tells the strip the frame is complete. The data leaves on PB5 (Arduino D11).

Boot order

1. At reset the CPU reads the stack pointer and reset address from the start of flash.
2. `startup_stm32h743xx.s` sets the stack and calls `SystemInit` in `clock.cpp` to get to 240 MHz.
3. It copies `.data` from flash to RAM, zeroes `.bss`, and runs the global constructors.
4. It calls `main()`, which calls `app::run()`.
5. `app::run()` sets up the driver and semaphore, creates the three tasks, and calls `vTaskStartScheduler()`. From then on FreeRTOS owns the CPU.

`initial_explanation.md` walks through these reset steps byte by byte for the earlier blink version, so it's a good companion read.

Memory layout

The linker script uses three RAM areas:

- **DTCM** (`0x20000000`), fast RAM next to the core, holds the main stack used before the scheduler starts and by interrupts.
- **AXI SRAM** (`0x24000000`) holds globals, task stacks and the two pixel frames.
- **D2 SRAM1** (`0x30000000`) holds the DMA transmit buffer, because the DMA engine can't reach DTCM. The buffer is 32-byte aligned so a later memory-protection setting can mark it uncached without moving it.